

Claimly App — Project Summary

Prepared by: Nino | Purpose: Overview of MVP progress for team review

Tech Stack

Layer	Technology
Frontend (mobile app)	Flutter (Dart) — Android device + Chrome (Flutter Web)
Backend (API)	Spring Boot 4.1.0, Java 21
Database	MySQL 8.0
Email delivery (OTP)	Gmail SMTP via Spring Boot Mail
Password security	BCrypt hashing

What's Been Built So Far

1. Onboarding & Authentication

- Landing page — app introduction, product overview (Income Protection, Excess Fee Cover), target audience messaging, Sign Up / Sign In entry points
- Sign Up — collects first name, last name, phone number, date of birth, SA ID number, email, and password (BCrypt-hashed before storage)
- Email OTP verification — 6-digit code generated and emailed on registration; user must verify before continuing
- Sign In — email + password authentication with proper error handling for wrong credentials or unverified accounts

2. Dashboard

- Personalized greeting using the user's first name
- Live cover status card — pulls the user's actual active policy from the database, or prompts to choose a plan if none is active
- Slide-out sidebar navigation — Cover, Claims, Claim History, Documents, Payments, Referrals, Support, Settings, Log Out
- Bottom navigation bar (Home / Cover / Claims / Profile)

3. Plan Selection & Policy Activation

- Browse available plans across both products (Income Protection, Excess Fee Cover), each with Basic/Standard tiers
- Review screen — plan summary, terms & conditions, required acceptance checkbox before proceeding
- Confirmation screen — generates a unique policy number (format: POL-YYYY-XXXXXX) on successful activation

- Activating a new plan automatically supersedes any previous active plan
- Dashboard reflects the newly activated policy immediately

Backend API Endpoints (so far)

Endpoint	Purpose
POST /api/register	Create account, send OTP
POST /api/verify-otp	Verify email with OTP code
POST /api/login	Authenticate user
POST /api/policies/subscribe	Activate a plan for a user
GET /api/policies/current/{email}	Fetch a user's active policy
GET /api/health	Basic server health check

Known Gaps / Not Yet Built

- Claims flow (file a claim, claim history, claim status tracking) — next planned feature
- Fingerprint/biometric login — in progress, paused partway through setup
- Selfie + ID document upload during registration — attempted, rolled back due to camera stability issues on the test device; deferred for a later pass
- Admin / document verification console — not started
- Session persistence (app currently requires signing in again after fully closing)
- Payments, notifications, and refer-a-friend are UI placeholders only

Notable Technical Decisions

- OTP delivery via email, not SMS/WhatsApp as originally specified in the requirements document — a deliberate MVP simplification, flagged as a deviation worth revisiting with the team
- Registration was simplified to progressive profiling (minimal fields first) partway through, then reverted to full-detail registration up front based on team direction
- Backend and Flutter frontend run locally during development (developer's PC), connected over the same Wi-Fi network

This summary reflects progress as of the current development session and will need updating as features are added.